O'REILLY®

RAG with AWS

About Me

- Senior Cloud AI Architect at DoiT.
- 10+ years experience in AWS, cloud architecture, and machine learning.
- Expert in RAG system design, optimization, and responsible AI practices.
- Developed 20+ production ready solution
- Goal: Helping enterprises deploy production-ready, grounded GenAI applications.

What is Generative AI?

Large Language Models (LLMs) are trained on massive datasets — billions of web pages, books, and documents — to generate text, images, and code with remarkable fluency. But this power comes with a fundamental constraint known as the **"closed-book" problem.**

Knowledge Cutoff

Models only know what they were trained on. Anything after the training cutoff date is invisible to the model — your latest product releases, regulatory changes, or internal updates simply don't exist to the LLM.

No Enterprise Data Access

LLMs have no access to your proprietary data — internal wikis, customer records, policy documents, or engineering runbooks. Without this context, responses are generic at best and dangerously wrong at worst.

Hallucination Risk

When a model lacks relevant knowledge, it doesn't say "I don't know" — it confidently fabricates plausible-sounding answers. This hallucination risk makes ungrounded LLMs unsuitable for production enterprise use cases.

The RAG Pattern

RAG solves the closed-book problem by combining semantic search with language generation. Instead of asking the model to recall facts from training, you **retrieve relevant context at query time** and inject it directly into the prompt — grounding every response in your actual data.

Content Database

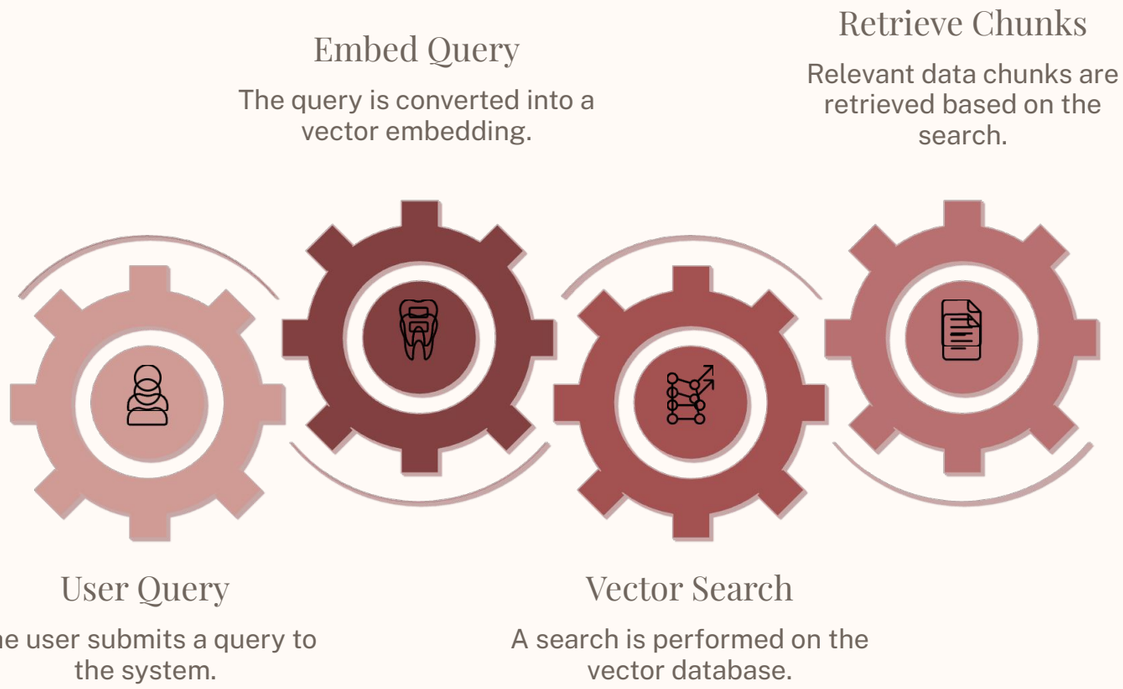
Enterprise documents chunked and stored as high-dimensional vector embeddings in a purpose-built vector store.

Retriever

Embeds the user query and performs semantic vector search to find the most contextually relevant document chunks.

Generator (LLM)

Receives the augmented prompt — original query plus retrieved chunks — and produces a grounded, cited response.



Key benefit: No retraining or fine-tuning required. Your data stays current and in your control.

Why RAG Over Fine-Tuning?

Both RAG and fine-tuning adapt LLM behavior, but they solve different problems. For most enterprise scenarios involving dynamic, proprietary data, **RAG is the right first choice** — faster to deploy, more cost-effective, and easier to keep current.

Aspect	RAG	Fine-Tuning
Data Freshness	Real-time updates — no retraining needed	Requires full retraining cycle for new data
Cost	Low — no GPU training compute required	High — compute-intensive, expensive at scale
Hallucination Control	Strong — citations tied to source documents	Moderate — hallucinations still possible
Setup Time	Hours to deploy and iterate	Days to weeks of training and evaluation
Data Privacy	Data stays in your VPC at all times	Data is used during the training process

Best Practice: Use RAG first. Fine-tune only when RAG alone isn't sufficient. Hybrid approaches that combine both techniques often deliver the best results for complex tasks.

RAG on AWS: The Bedrock Ecosystem

Amazon Bedrock Knowledge Bases

Fully managed RAG — zero infrastructure to configure or maintain.

Automated ingestion from **S3, Confluence, SharePoint, Salesforce, Web Crawler**

- Automatic chunking, embedding generation, and vector storage

Retrieve API — semantic search only

RetrieveAndGenerate API — search + LLM response in one call

- Built-in citations back to source documents

Supported Vector Stores

- **Amazon OpenSearch Serverless**
  Fully managed, scalable semantic search
- **Amazon Aurora PostgreSQL**
  pgvector extension for relational + vector
- **Amazon S3 Vectors**
  NEW — up to 90% cost reduction
- **Amazon Neptune Analytics**
  Graph + vector for connected knowledge
- **MongoDB Atlas, Pinecone, Redis**
  Third-party vector store integrations

What's New in Bedrock (2025–2026)

LATEST UPDATES

AWS has shipped a wave of major capabilities that significantly expand what you can build on Bedrock. These updates directly impact how you architect and scale RAG solutions in production.

Amazon S3 Vectors

First cloud object store with **native vector support**. Up to **90% cheaper** vector storage with sub-second queries, natively integrated with Bedrock Knowledge Bases.

Amazon Nova 2 Models

Nova 2 Lite and Nova 2 Pro with **extended thinking**, **1M token context window**, built-in code interpreter and web grounding for deeper, more accurate reasoning.

Multimodal RAG

Native support for **video, audio, images, and text** in a unified knowledge base using Nova Multimodal Embeddings — go beyond document search.

Multi-Agent Collaboration

Generally available: networks of **specialized agents** coordinated by a supervisor agent for complex, multi-step enterprise workflows.

Project Mantle

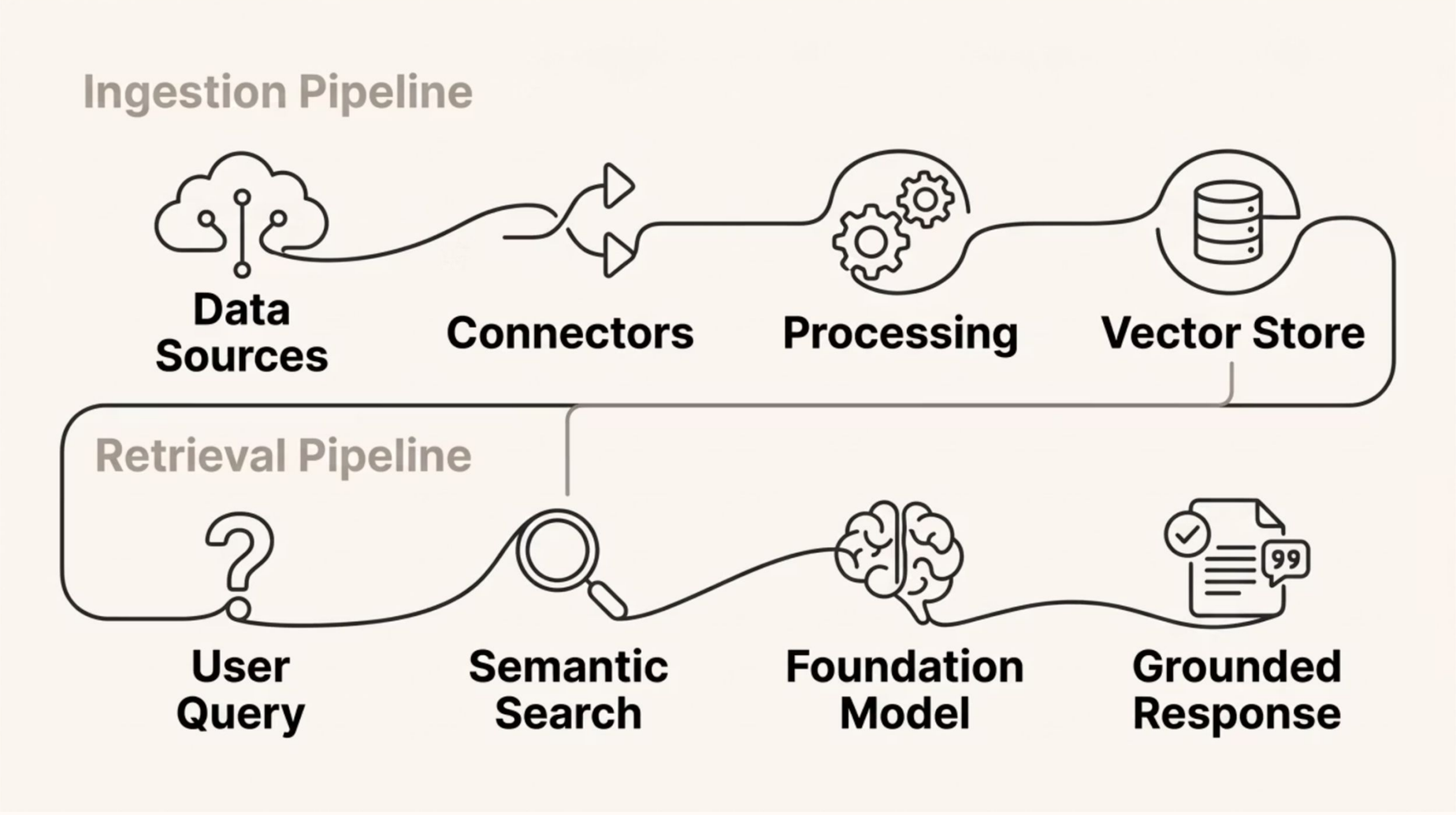
New distributed inference engine for open-weight models including **DeepSeek, Gemma, Mistral, and Qwen** — run cutting-edge OSS models on Bedrock infrastructure.

102+ Foundation Models

Access to over **102 foundation models from 17 providers** — including Anthropic, Meta, Mistral, Stability AI, and more — through a single unified API.

Bedrock Knowledge Base Architecture

The Bedrock Knowledge Base architecture separates concerns cleanly into an **ingestion pipeline** and a **retrieval pipeline**. Understanding both flows is essential for designing production RAG systems.



Two distinct pipelines: Ingestion runs asynchronously to keep your knowledge base current. Retrieval runs synchronously at query time — typically sub-second with the right vector store configuration.

RAG with Bedrock Agents

Bedrock Agents elevate RAG from **search-and-respond** to **search-reason-act**. An agent can retrieve relevant knowledge, reason over it, and then take real actions — calling APIs, running code, or delegating to specialist sub-agents — all within a single orchestrated workflow.

Action Groups

Define API schemas and Lambda functions the agent can invoke based on user intent. The agent autonomously selects and calls the right action.

Guardrails

Content filters, denied topic policies, prompt injection detection, and automated reasoning checks — applied to both inputs and outputs.

Memory

Session context is retained across multi-turn conversations, enabling natural, coherent dialogues without re-stating prior context.

Multi-Agent Collaboration

A supervisor agent orchestrates networks of specialized sub-agents — each with its own knowledge base and tools — for complex enterprise workflows.

Example use case: An insurance agent retrieves policy documents via Knowledge Base, validates coverage rules, then executes a claim submission action via Lambda — end to end, with citations and guardrails applied throughout.

Key Takeaways

RAG is the fastest, most cost-effective path to grounding LLM responses in your enterprise data. Here's what you should walk away knowing:

RAG = Grounded Responses

Retrieve, don't recall. RAG grounds every LLM response in your real data — no retraining, no fine-tuning required.

Bedrock KB is Fully Managed

No infrastructure to configure. Automated ingestion, chunking, embedding, and vector storage out of the box.

S3 Vectors = Cost Leader

The newest and most cost-effective vector store option — up to 90% cheaper with sub-second query performance.

Nova 2: Extended Context

Nova 2 models bring extended thinking and a 1M token context window — enabling deeper reasoning over large document sets.

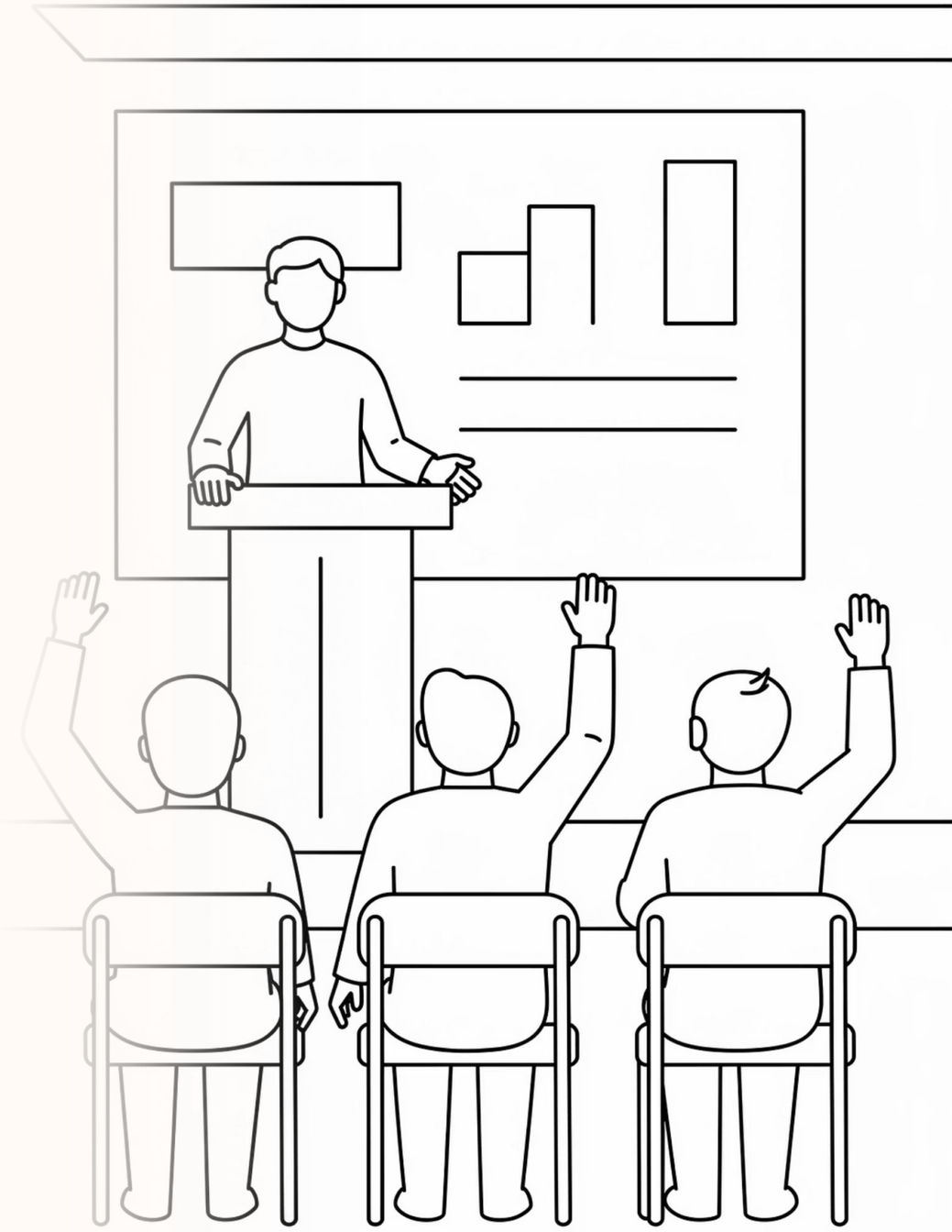
Multimodal RAG is Here

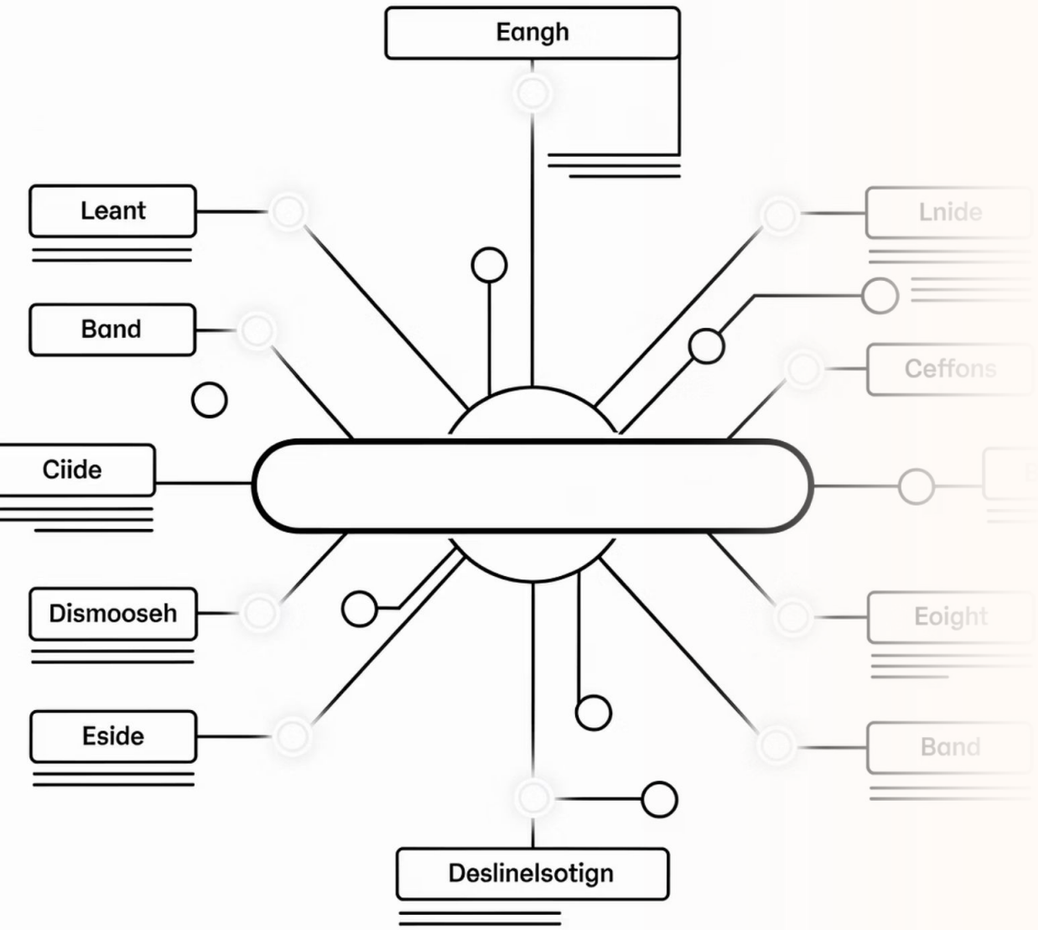
Go beyond text. Bedrock now supports video, audio, and image inputs natively in your knowledge base.

Always Add Guardrails

Production RAG requires safety layers. Always pair your Knowledge Base and Agents with Bedrock Guardrails.

Q&A





Building Semantic Search with OpenSearch and Vector Databases

This module explores how modern search systems move beyond keyword matching to understand the **meaning** behind queries — enabling dramatically more relevant retrieval for RAG pipelines, Q&A systems, and production search applications.

From Keywords to Meaning

❌ The Problem with Keyword Search

"Cheap flights to NYC" won't match **"affordable airfare to New York"** — even though they mean exactly the same thing.

- Misses synonyms, paraphrases, and user intent
- No understanding of context or negation
- Fails on natural language questions
- Term frequency, not meaning, drives ranking

✅ Semantic Search Solves This

Semantic search converts text into **vector embeddings** — dense numerical representations that capture meaning. Both queries and documents live in the same vector space, so similarity is measured by geometric distance.

- Cosine similarity, dot product, or Euclidean (L2)

"Cheap flights to NYC" and "affordable airfare to New York" become **close neighbors**

- Handles synonyms, paraphrases, and intent naturally

What Are Vector Embeddings?

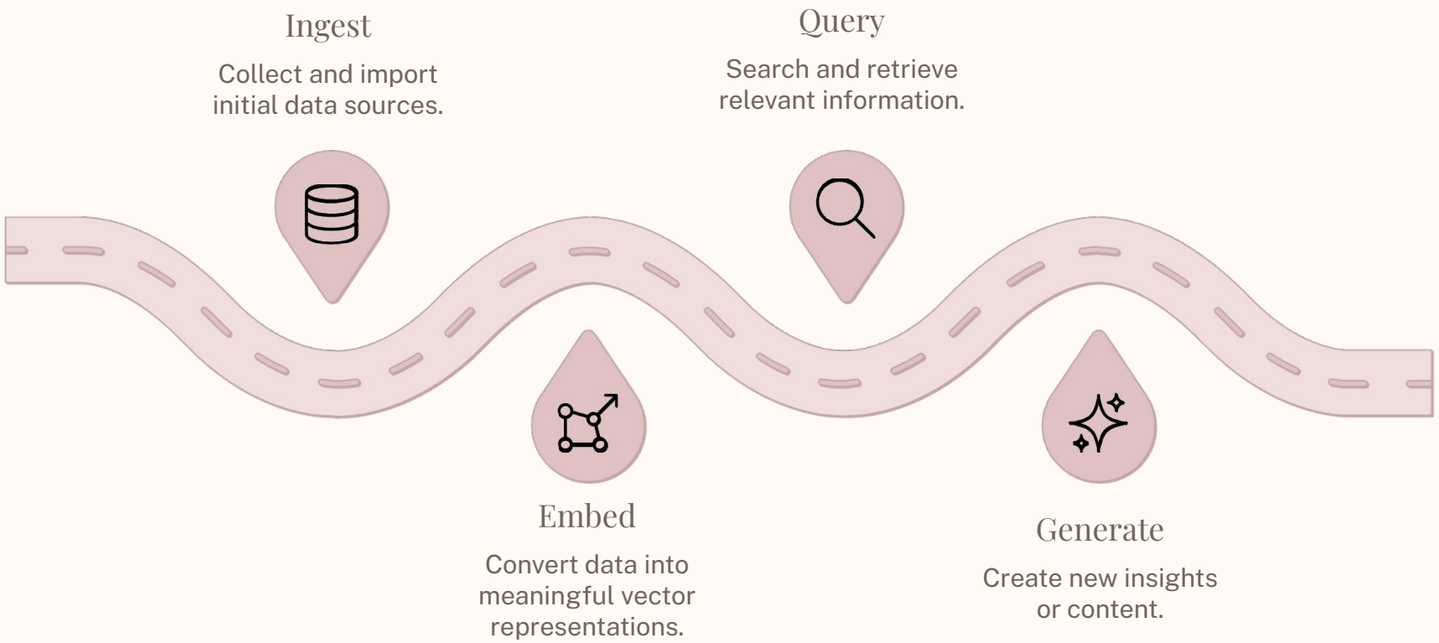
An embedding model transforms any piece of text — or image, audio, or video — into a **high-dimensional numerical vector**. Semantically similar content produces vectors that cluster together in this space, enabling geometric search.

Example: "The cat sat on the mat" → `[0.023, -0.156, 0.891, ..., 0.042]` (1024 dimensions). Each dimension encodes abstract features: topic, sentiment, entity type, and more.

Model	Dimensions	Modality	Languages	Highlight
Amazon Titan Text V2	256, 512, 1024	Text	100+	AWS-native, cost-effective
Amazon Titan Multimodal G1	256, 384, 1024	Text + Image	English	Image-text joint space
Amazon Nova Multimodal	Configurable	Text + Image + Video + Audio	Multi	Full media support
Cohere Embed v4 ⭐	256–1536	Text + Image + Documents	100+	Natively handles tables, graphs, code, handwriting
Cohere Embed Multilingual	1024	Text	100+	Cross-lingual retrieval

Cohere Embed v4 (NEW): Natively processes complex documents with tables, graphs, diagrams, code snippets, and handwritten notes — no preprocessing pipeline needed.

Vector Search: How It Works



Vector search operates in two distinct phases: an **offline ingestion** pipeline that processes and stores document embeddings, and a **runtime query** pipeline that embeds the user's question and retrieves the closest matches before passing them to the LLM.

Search Algorithms

Exact k-NN

Brute force, 100% recall — best for small datasets (<100K vectors)

Approximate k-NN (ANN)

HNSW algorithm — fast at scale, ~95–99% recall on millions of vectors

Distance Metrics

Cosine Similarity

Most common — measures angle between vectors, scale-invariant

Dot Product / Euclidean

Dot product for normalized vectors; L2 for absolute distance

Vector Database Options on AWS

Vector Store	Type	Best For	Key Feature
OpenSearch Serverless	Managed serverless	Production RAG, hybrid search	Auto-scaling, zero infrastructure management
OpenSearch Service	Managed cluster	Enterprise-grade, full control	GPU-accelerated indexing, FAISS/NMSLIB/Lucene engines
Amazon S3 Vectors ⭐	Object storage	Cost-optimized, large-scale cold storage	Up to 90% cheaper, sub-second queries, 2B vectors/index
Aurora PostgreSQL + pgvector	Relational DB	Existing PostgreSQL users	SQL-based vector queries, familiar tooling
Amazon Neptune Analytics	Graph DB	Knowledge graphs + vectors	Graph traversals combined with vector search
Amazon MemoryDB	In-memory DB	Ultra-low latency use cases	Microsecond reads for real-time retrieval

S3 Vectors (NEW): First cloud object store with native vector support. Ideal for infrequent-query workloads and tiered storage — S3 Vectors for cold, OpenSearch for hot.

OpenSearch GPU Indexing (NEW): Build optimized vector indexes up to **10x faster** at **1/4 the cost**. Auto-optimize jobs tune index configuration automatically.

Keyword vs. Semantic vs. Hybrid Search

Aspect	Keyword (Lexical)	Semantic (Vector)	Hybrid
How it works	BM25 term matching	Vector cosine similarity	Both combined, normalized scores
Strengths	Exact matches, product IDs, codes	Natural language, synonyms, intent	Best of both worlds
Weaknesses	Misses synonyms and context	Can miss exact terms and codes	More complex to configure
Use case	SKU lookup, log search	Q&A, document search	Production RAG pipelines

Hybrid Search on AWS

OpenSearch Serverless

Set `overrideSearchType: HYBRID` in Bedrock KB config — one line change

OpenSearch Service

Parallel query processing with conditional scoring logic for fine-grained control

Aurora / MongoDB Atlas

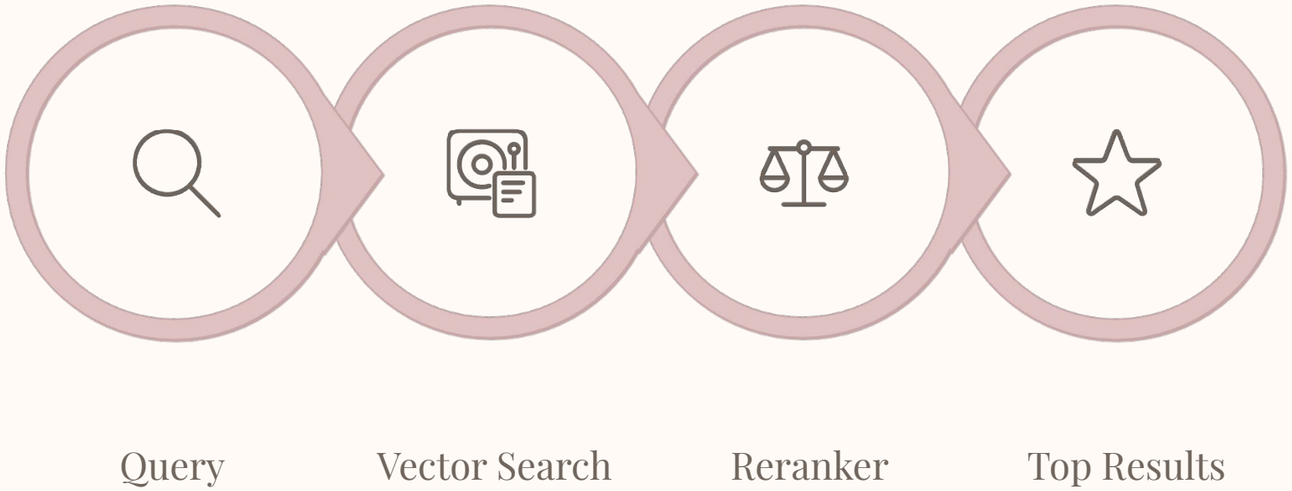
Both now support hybrid search natively with Bedrock Knowledge Bases

Auto Semantic Enrichment ⭐

Zero-config sparse vector enrichment — **+20% relevance** (English), **+105%** (multilingual)

Reranking: The Second Pass

Initial vector retrieval (top-K) is optimized for **speed**, not precision. A reranker model reads the actual query and each candidate chunk together, producing a much more accurate relevance score — without the cost of sending all 50 results to the LLM.



Reranking Models on Bedrock

Cohere Rerank 3.5

Cross-encoder relevance scoring — reorders results with deep query-document understanding

Amazon Titan Reranker

Native Bedrock reranking — no external API calls, tight integration

How to Enable

Add `rerankingConfiguration` to your `KnowledgeBaseVectorSearchConfiguration` in the `Retrieve` or `RetrieveAndGenerate` API call — or toggle it in the Bedrock console.

Key benefit: Higher precision without increasing the number of chunks sent to the LLM — saving both tokens and inference cost.

Chunking Strategies

How you split documents **directly determines retrieval quality**. Chunks that are too large dilute relevance; chunks that are too small lose context. The right strategy depends on document structure and query type.

Strategy	How It Works	Best For	Bedrock Support
Fixed-size	Split every N tokens with configurable overlap	Simple, uniform documents	✅ Default
Hierarchical	Parent (large) + child (small) chunks; retrieve child, return parent for context	Long technical docs, manuals	✅ Yes
Semantic	NLP-based splitting at natural topic boundaries	Mixed-topic documents	✅ Yes
None	Entire file treated as one chunk	Pre-processed or short documents	✅ Yes

Bedrock defaults: ~300 tokens per chunk with sentence preservation enabled.

Best Practices

Start with fixed-size

300–500 tokens with 10–20% overlap — covers most use cases

Use hierarchical for long docs

Technical manuals, legal documents, research papers

Add metadata filters

Filter by date, category, or source to narrow retrieval scope and improve precision

Keyword vs. Semantic Retrieval Trade-offs

Take 5 minutes to discuss these questions with your table. There are no trick answers — the goal is to surface real design trade-offs you'll face in production systems.

1

When does keyword win?

Exact product codes, SKU lookups, structured log analysis — anywhere precision on known terms matters

2

When does semantic shine?

Natural language Q&A, multilingual queries, paraphrased questions — intent over exact terms

3

Why is hybrid the default?

Covers both exact and fuzzy matches — no single approach dominates all query types in production

4

What are the cost levers?

Embedding compute, vector storage pricing, reranking inference — where do you optimize?

5

Chunking & retrieval quality?

How does chunk size affect precision vs. recall? When does hierarchical chunking help?

6

S3 Vectors vs. OpenSearch?

Infrequent queries → S3 Vectors. High-frequency production RAG → OpenSearch Serverless

Key Takeaways

Semantic = Meaning

Vector embeddings capture semantic meaning — not just token overlap. Queries and documents share the same geometric space.

Hybrid is the Default

Keyword + semantic combined is the production best practice. Neither approach alone covers all real-world query types.

Right Store for the Job

OpenSearch Serverless for hot production RAG; S3 Vectors for cost-optimized cold storage at massive scale.

Cohere Embed v4

Natively handles complex multimodal documents — tables, graphs, code, handwriting — with no preprocessing pipeline.

Reranking Boosts Precision

Cohere Rerank 3.5 improves result quality without increasing the number of chunks sent to the LLM — saving tokens and cost.

Chunking Strategy Matters

Start with fixed-size (300–500 tokens, 20% overlap). Iterate based on evaluation. Use hierarchical for long technical docs.

Auto Semantic Enrichment

OpenSearch Serverless gives you sparse vector enrichment with zero configuration — up to 105% relevance lift for multilingual queries.

Questions & Discussion

Bedrock GenAI Integration

Amazon Bedrock API Landscape

Two primary ways to invoke foundation models. The right choice depends on your use case — but for most scenarios, the recommendation is clear.

API	Use Case	Key Benefit
InvokeModel	Single prompt, model-specific payload	Direct control, model-native parameters
Converse / ConverseStream	Multi-turn chat, tool use, guardrails	Model-agnostic — write once, swap models

Use the Converse API. It's the recommended approach for nearly every use case.

Model-Agnostic Format

Consistent request/response structure across Claude, Nova, Llama, Mistral, Cohere, DeepSeek, and more.

Built-In Capabilities

Native support for tool use, guardrails, documents, images, and video — no extra wiring needed.

Streaming Ready

ConverseStream delivers real-time token delivery for chatbots and live demos with minimal code changes.

Your First Bedrock Call (Python)

The following snippet shows a complete, working call to Claude using the Converse API. It retrieves a response and prints token usage — essential for cost monitoring.

```
import boto3, json

bedrock = boto3.client("bedrock-runtime", region_name="us-east-1")
# Simple text generation with Converse API
response = bedrock.converse(
    modelId="us.anthropic.claude-sonnet-4-5-20250514-v1:0",
    messages=[
        {
            "role": "user",
            "content": [{"text": "Explain RAG in 3 sentences."}]
        }
    ],
    inferenceConfig={
        "maxTokens": 512,
        "temperature": 0.3,
        "topP": 0.9
    }
)
print(response["output"]["message"]["content"][0]["text"])
print(f"Tokens: {response['usage']['inputTokens']} in / {response['usage']['outputTokens']} out")
```

temperature (0–1)

Lower = deterministic. Higher = creative. Start at 0.3 for factual tasks.

topP (0–1)

Nucleus sampling — controls output diversity. Combine with temperature carefully.

maxTokens

Hard cap on response length. Always set to avoid runaway costs.

stopSequences

Custom strings that halt generation — useful for structured output.

Streaming Responses with ConverseStream

For real-time UX — chatbots, live demos, and progressive output — use `converse_stream`. Tokens are delivered as they're generated, eliminating the wait for a full response.

```
response = bedrock.converse_stream(
    modelId="us.anthropic.claude-sonnet-4-5-20250514-v1:0",
    messages=[
        {"role": "user", "content": [{"text": "Write a haiku about cloud computing."}]}
    ],
    inferenceConfig={"maxTokens": 256, "temperature": 0.7}
)

for event in response["stream"]:
    if "contentBlockDelta" in event:
        print(event["contentBlockDelta"]["delta"]["text"], end="", flush=True)
```

Why Streaming Matters

Users perceive streaming interfaces as ~3× faster than batch responses — even when total generation time is identical. It's a UX multiplier for any interactive app.

Event Stream Structure

The stream emits different event types: `messageStart`, `contentBlockDelta`, `messageStop`, and `metadata`. Filter by `contentBlockDelta` to extract text tokens in real time.

Multi-Turn Conversations

The Converse API maintains conversational state through the `messages` array — you append each turn and pass the full history on every call. The system prompt locks in the model's persona and behavior constraints.

```
messages = []

def chat(user_input):
    messages.append({"role": "user", "content": [{"text": user_input}]})
    response = bedrock.converse(
        modelId="us.amazon.nova-pro-v1:0",
        system=[{"text": "You are a helpful AWS solutions architect."}],
        messages=messages,
        inferenceConfig={"maxTokens": 1024, "temperature": 0.3}
    )
    assistant_msg = response["output"]["message"]
    messages.append(assistant_msg)
    return assistant_msg["content"][0]["text"]

print(chat("What is Amazon Bedrock?"))
print(chat("How does it compare to SageMaker for inference?"))
```

System Prompt Best Practice: Be explicit about role, constraints, and output format. A well-crafted system prompt reduces hallucinations and improves response consistency across all turns.

Applying Guardrails at Inference Time

Guardrails plug directly into the Converse API — a single config block activates all your safety policies with zero extra latency in the critical path.

```
response = bedrock.converse(
    modelId="us.anthropic.claude-sonnet-4-5-20250514-v1:0",
    messages=[{"role": "user", "content": [{"text": user_input}]}],
    guardrailConfig={
        "guardrailIdentifier": "<GUARDRAIL_ID>",
        "guardrailVersion": "DRAFT"
    },
    inferenceConfig={"maxTokens": 512}
)
```

Filter	What It Does
Content filters	Block hate, insults, sexual, violence, misconduct (text + images)
Prompt attacks	Detect jailbreaks, prompt injection, prompt leakage
Denied topics	Block specific topics you define per application
Word filters	Block custom words and phrases
Sensitive info (PII)	Block or mask names, SSNs, emails, and more
Contextual grounding	Detect hallucinations not grounded in source data
Automated reasoning	Validate responses against logical rules and policies

New: Standard tier now extends protection to harmful content within code elements. High-accuracy automated reasoning checks are also available.

Cross-Region Inference

Scale globally without changing your application logic — just swap the model ID prefix. Intelligent routing considers availability, capacity, and latency automatically. All logs remain in your source region.

```
# Geographic — stays within a region group (e.g., US)
response = bedrock.converse(
    modelId="us.anthropic.claude-sonnet-4-5-20250514-v1:0",   # geographic profile
    messages=[...]
)

# Global — routes to any AWS commercial region worldwide (~10% cheaper)
response = bedrock.converse(
    modelId="global.anthropic.claude-sonnet-4-5-20250514-v1:0",  # global profile
    messages=[...]
)
```

Type	Scope	Cost	Best For
Single region	One region	Standard	Strict data residency requirements
Geographic CRIS	US / EU / APAC	Standard	Compliance + throughput
Global CRIS	All commercial regions	~10% savings	Max throughput + cost optimization

Prompt Management

Treat prompts like code — version them, test variants, and share across teams. Bedrock's Prompt Management API stores prompts as first-class artifacts with full lifecycle support.

```
bedrock_agent = boto3.client("bedrock-agent")

prompt = bedrock_agent.create_prompt(
    name="rag-answer-prompt",
    description="Generate answers from retrieved context",
    defaultVariant="v1",
    variants=[
        {
            "name": "v1",
            "templateType": "TEXT",
            "templateConfiguration": {
                "text": {
                    "text": "Answer the question based on the context.\n\nContext: {{context}}\n\nQuestion: {{question}}\n\nAnswer:",
                    "inputVariables": [{"name": "context"}, {"name": "question"}]
                }
            },
            "modelId": "us.anthropic.claude-sonnet-4-5-20250514-v1:0",
            "inferenceConfiguration": {
                "text": {"maxTokens": 512, "temperature": 0.2}
            }
        }
    ]
)
```

Version Control

Create named snapshots and roll back to any previous version instantly.

A/B Testing

Define multiple variants and test which prompt drives the best outcomes.

Team Sharing

Publish prompts as shared resources — consistent behavior across all services.

Flows Integration

Plug managed prompts directly into Bedrock Flows for multi-step orchestration.

Model Selection Guide

Match the model to the task. Over-engineering with a flagship model for routing tasks wastes budget; under-engineering complex orchestration leads to failures.

Model	Strengths	Best For	Cost
Claude Opus 4.6	Most capable reasoning	Complex analysis, orchestration	$$$$
Claude Sonnet 4.5/4.6	Strong balance	Agents, coding, general tasks	$$$
Claude Haiku 4.5	Fast, cheap	High-volume, latency-sensitive	$
Amazon Nova Pro	Multimodal, balanced	RAG, document analysis	$$
Amazon Nova Lite	Low-cost multimodal	Visual Q&A, simple tasks	$
Amazon Nova Micro	Fastest text-only	Summarization, classification	$
Nova 2 Lite	Extended thinking, 1M context	Complex reasoning, code	$$
DeepSeek R1/V3.2	Strong reasoning	Math, code, analysis	$$
Llama 4 Maverick/Scout	Open-weight, versatile	General purpose	$$
Mistral Large 3	Coding, multilingual	European compliance	$$

Strategy: Use Haiku / Nova Micro for routing and classification. Sonnet / Nova Pro for generation. Opus for complex orchestration where accuracy is non-negotiable.

Key Takeaways

Converse API is your default

Model-agnostic, supports all features — tools, guardrails, streaming, documents. Write once, swap models freely.

ConverseStream for interactive UX

Real-time token delivery dramatically improves perceived performance in chatbots and live interfaces.

Tool use = the agent building block

Models orchestrate external function calls — stack tools to build powerful, multi-step agentic workflows.

Guardrails are non-negotiable in production

Content filters, PII masking, grounding checks, and automated reasoning protect every inference call.

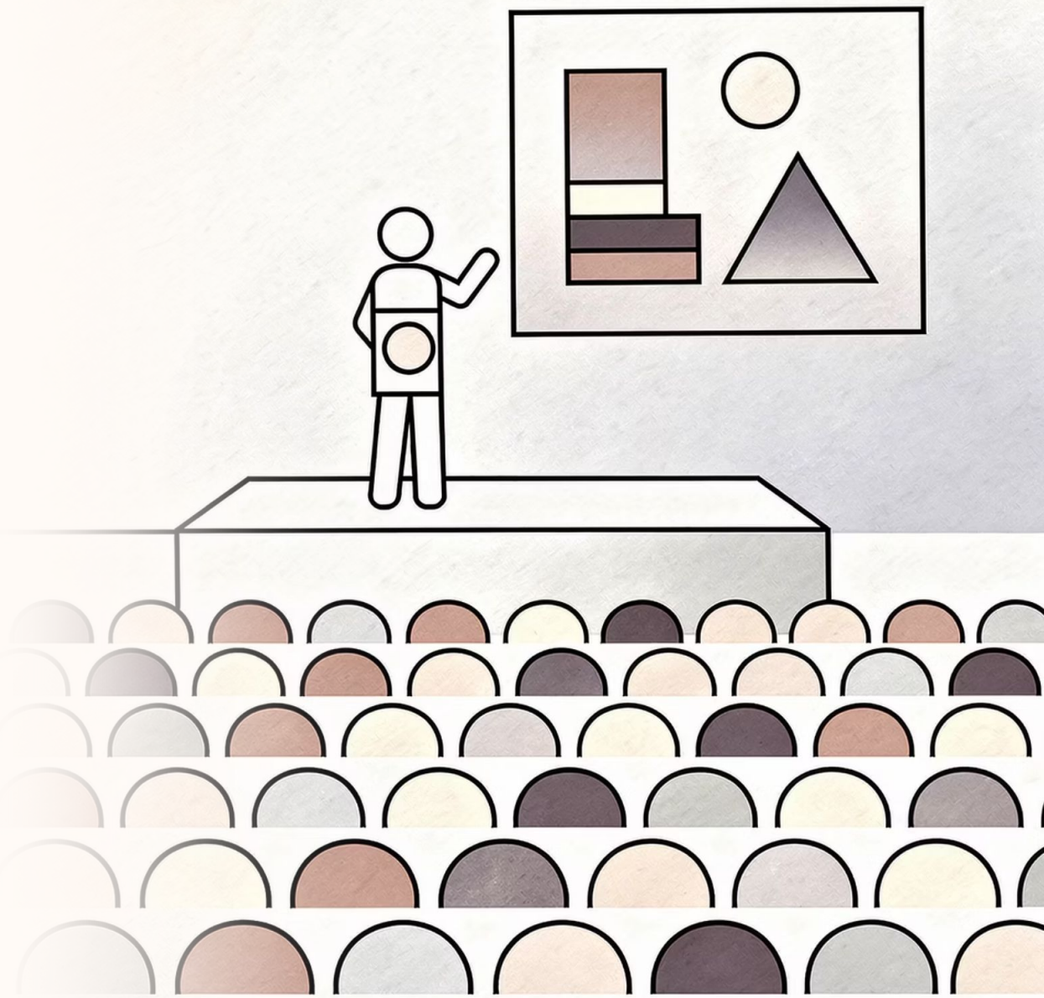
Global CRIS = ~10% savings, one line change

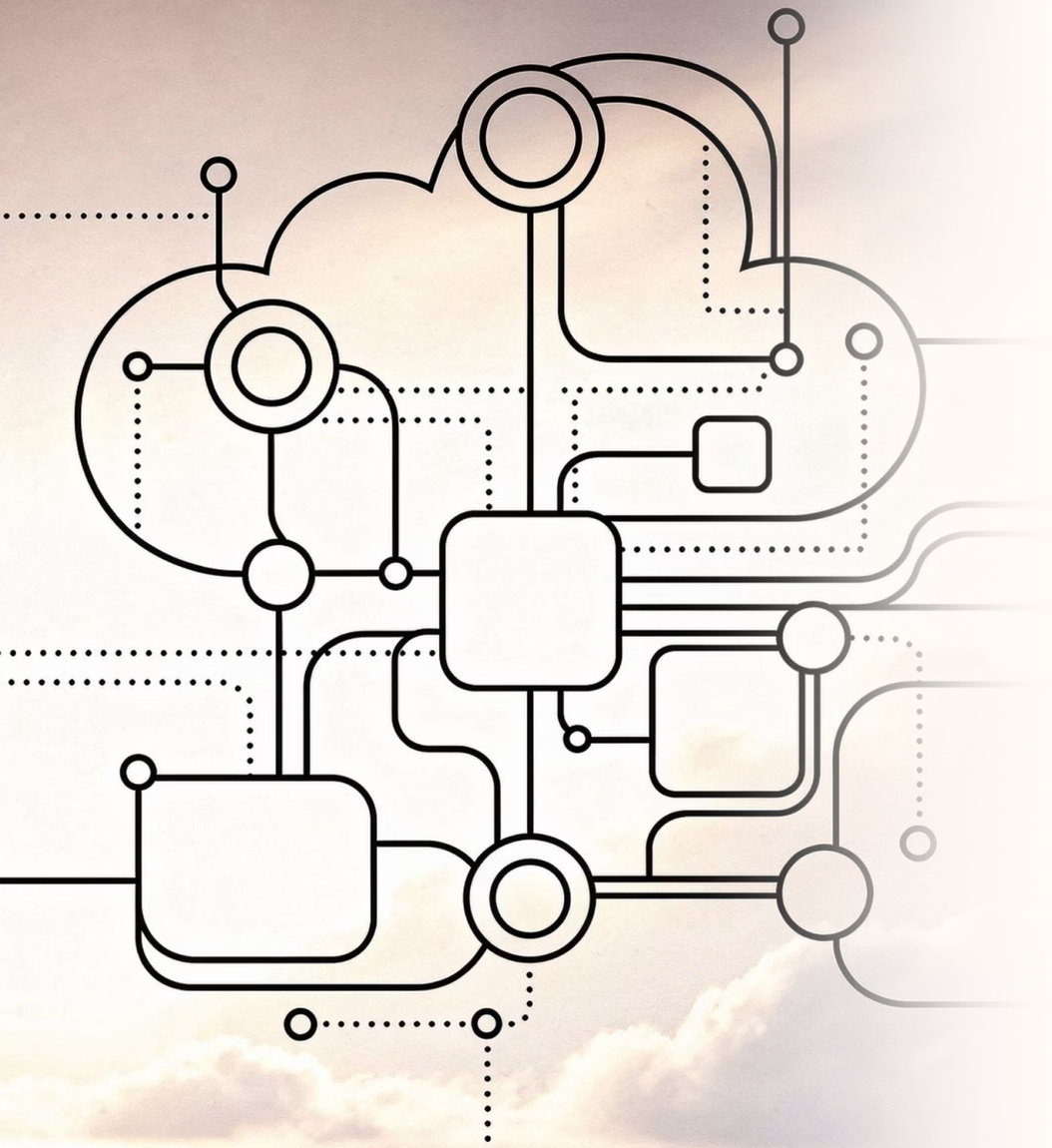
Swap `us.` for `global.` in your model ID for intelligent worldwide routing at lower cost.

Model selection is a cost lever

Cheap/fast for routing, capable for generation, powerful for orchestration — right-size every call.

Q&A





Building End-to-End RAG Pipeline

RAG Architecture Patterns

Choose your pattern based on control requirements, workflow complexity, and time-to-value. All three run on AWS Bedrock.

1

Basic RAG — Managed

Flow: User → Bedrock KB (RetrieveAndGenerate) → Response + Citations

- Single API call handles both retrieval and generation
- Bedrock manages chunking, embedding, and prompt assembly

Best for: Quick POCs, simple Q&A use cases

2

Custom RAG — Decoupled

Flow: User → Retrieve API → Custom prompt assembly → Converse API → Response

- Separate retrieval and generation steps give full prompt control
- Supports multi-source retrieval and complex business logic

Best for: Custom prompts, multi-source retrieval, complex logic

3

Agentic RAG — Autonomous

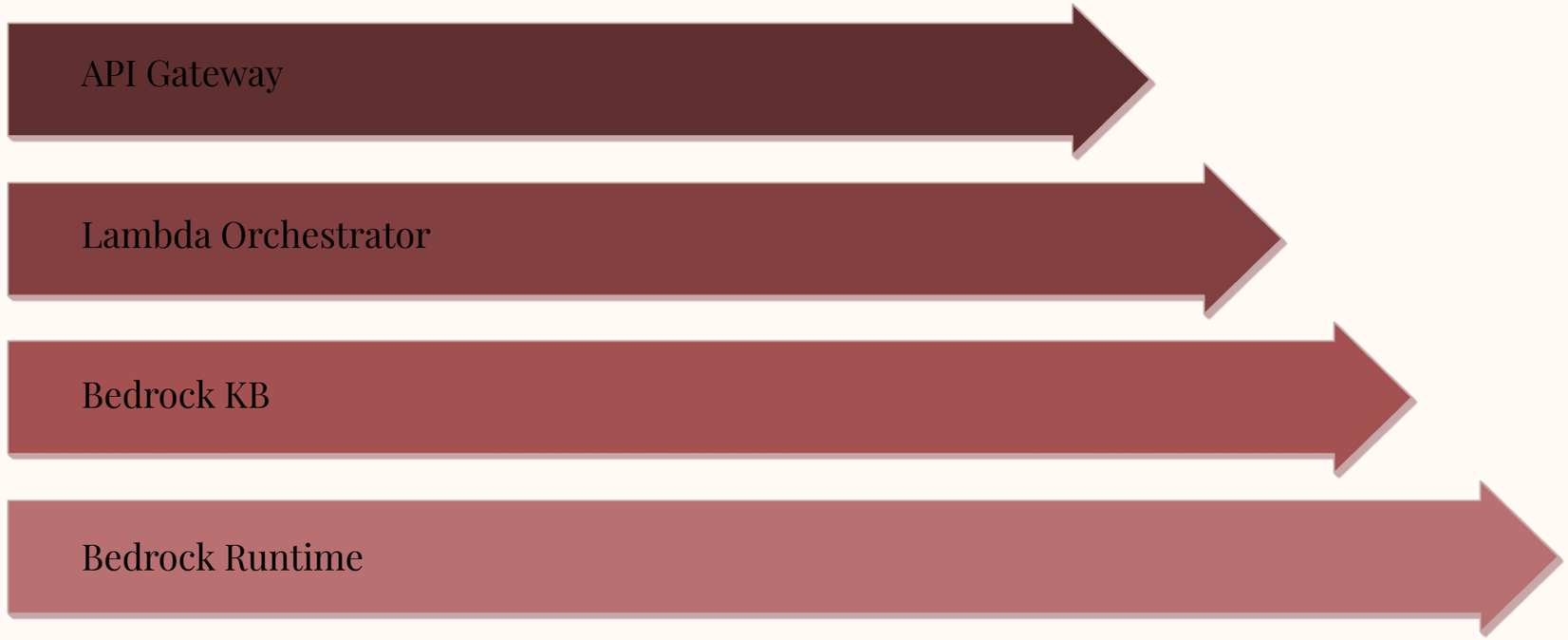
Flow: User → Bedrock Agent → [KB query | Action Group | Code Interpreter] → Response

- Agent reasons autonomously: when to retrieve, which tools to call
- ReAct loop: observe → reason → act → repeat

Best for: Complex workflows, multi-step tasks, dynamic tool selection

Serverless RAG Architecture

A fully managed, event-driven stack built on API Gateway, Lambda, and Bedrock — no servers to provision or patch.



Request Layer

API Gateway — REST or WebSocket for streaming; handles auth, throttling, WAF

Lambda — orchestration logic, prompt assembly, error handling (Python)

Bedrock Layer

Bedrock KB — managed retrieval with hybrid search + reranking

Bedrock Runtime — generation via Converse API

Guardrails — content filtering, PII masking, grounding checks

S3 / OpenSearch Serverless — document store and vector index

Bedrock Agents: Agentic RAG

Agents extend RAG with autonomous reasoning. They decide *when* to retrieve, *which* tools to invoke, and *how* to chain steps — all driven by the ReAct orchestration pattern.

Build-Time Components

Foundation Model

Reasoning engine — Claude Sonnet, Nova Pro

Instructions

Natural language description of agent behavior and constraints

Knowledge Bases

One or more KBs for grounded retrieval

Action Groups

API calls the agent can make via Lambda or function schemas

Guardrails & Memory

Safety filters + session context for multi-turn conversations

Runtime Flow — ReAct Pattern

1. **Pre-processing**

   Categorize and contextualize user input; apply guardrails

2. **Orchestration**

   Reason → select tool → execute → observe → repeat until done

3. **Post-processing**

   Format final response; apply output guardrails and citations

```
bedrock_agent.create_agent(
 agentName="rag-assistant",
 foundationModel="anthropic.claude-sonnet-4-5-...",
 instruction="Answer questions about company policies.
 Use the KB. Always cite sources."
)
```

Bedrock Flows: Visual Workflow Orchestration

Flows let you build multi-step RAG pipelines as visual graphs — drag, connect, and deploy without managing orchestration code.

Available Node Types

Prompt — invoke any Bedrock model

Knowledge Base — retrieve or retrieve-and-generate

Agent — invoke a Bedrock Agent

Lambda — custom logic

Condition — branching logic

Iterator / Collector — loop over arrays

DoWhile Loop — iterate until condition met

Inline Code, Lex, S3 — code snippets, intent detection, storage

Example: Document Q&A with Routing

→ Input ↠ Condition Node

Topic classifier routes to the right KB

→ "policy" ↠ Policy KB ↠ Prompt Node

Retrieves policy docs, generates grounded answer

→ "technical" ↠ Tech KB ↠ Prompt Node

Retrieves technical docs with specialized prompt

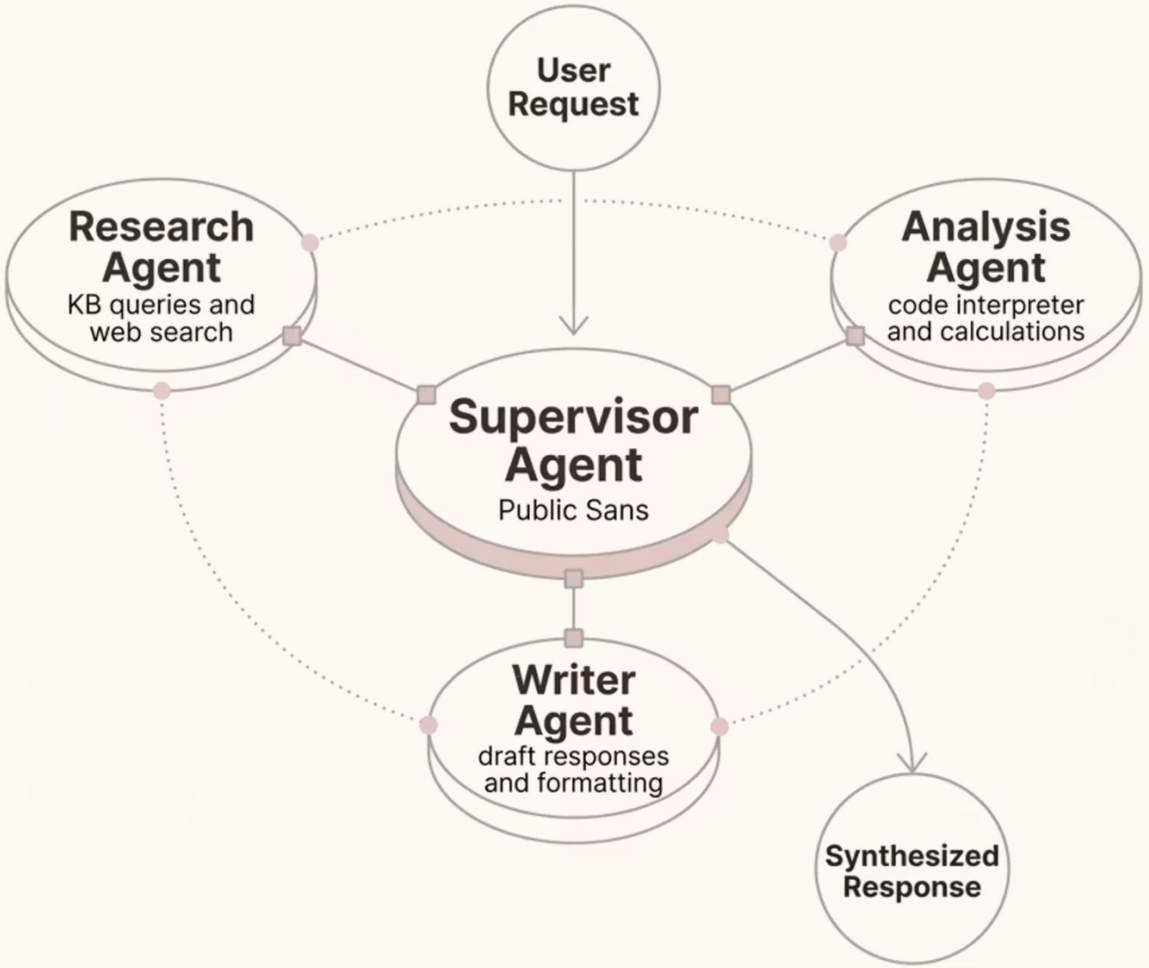
→ "other" ↠ General Prompt Node ↠ Output

Falls back to general knowledge response

Flows also support guardrails on prompt and KB nodes, real-time execution visibility, and DoWhile loops for iterative refinement.

Multi-Agent Collaboration

For complex, multi-domain workflows, decompose responsibilities across a **supervisor agent** that coordinates specialized sub-agents — each optimized for a distinct task.



Inline Agents
Dynamically adjust agent roles and instructions at runtime without redeployment

Payload Referencing
Supervisor references linked data instead of embedding it — reduces latency and cost

IaC Support
CloudFormation and CDK support for templating reusable, version-controlled agent teams

Monitoring
Track, debug, and optimize agent interactions with built-in trace and CloudWatch integration

RAG Evaluation with Bedrock

Bedrock's built-in evaluation (GA) gives you a quantitative quality signal across the full RAG pipeline — retrieval, generation, and responsible AI — without building your own eval harness.

Retrieval Metrics

Context relevance — are retrieved chunks relevant to the query?
Context coverage — do chunks cover all aspects of the answer?

Generation Metrics

- Correctness, Completeness, Helpfulness, Logical coherence

Faithfulness — is the response grounded? (hallucination detection)
Citation precision / coverage — accurate and complete citations?

Responsible AI Metrics

- Harmfulness, Stereotyping, Refusal rate

Running an Evaluation Job

```
eval_job = bedrock.create_evaluation_job(
  jobName="rag-eval-v1",
  applicationType="RagEvaluation",
  evaluationConfig={
    "automated": {
      "datasetMetricConfigs": [{
        "taskType": "Custom",
        "metricNames": [
          "Builtin.Correctness",
          "Builtin.Completeness",
          "Builtin.Faithfulness",
          "Builtin.ContextRelevance"
        ],
        "dataset": {
          "s3Uri": "s3://bucket/eval.jsonl"
        }
      }]
    }
```

Also supports custom RAG pipelines — bring your own input/output pairs and retrieved contexts for fully flexible evaluation.

Advanced RAG Techniques

Move beyond basic vector search with these production-proven techniques. Each addresses a specific retrieval or generation quality gap.

Technique	What It Does	When to Use
Hierarchical chunking	Parent/child chunks — retrieve child, return parent for full context	Long technical documents
Semantic chunking	Split at topic boundaries using NLP rather than fixed token counts	Mixed-topic documents
Metadata filtering	Filter by date, category, or source before vector search	Large document collections
Hybrid search	Combine keyword + semantic retrieval for broader recall	Production RAG (recommended default)
Reranking	Second-pass scoring with Cohere Rerank 3.5 to boost precision	When top-K precision matters
Query decomposition	Break complex queries into focused sub-queries, merge results	Multi-part questions
HyDE	Generate a hypothetical answer, embed it, search for similar docs	Abstract or under-specified queries
Multimodal RAG	Index and retrieve text, images, audio, and video	Rich media content
Graph RAG	Neptune Analytics — graph traversals combined with vector search	Entity relationships and knowledge graphs

Production Deployment Checklist

Production readiness is not a single feature — it's the intersection of infrastructure hardening, pipeline quality, safety controls, observability, and cost discipline.

Infrastructure

- API Gateway with throttling, WAF, and API keys
- Lambda: 512MB–1GB memory, 30s+ timeout
- VPC endpoints (PrivateLink) for Bedrock
- S3 with versioning and encryption

RAG Pipeline

- Knowledge Base with appropriate chunking strategy
- Hybrid search enabled; reranking configured
- Metadata filters for scoping results

Safety & Quality

- Guardrails: content filters, PII masking, grounding checks
- Evaluation job with baseline metrics established
- System prompt with clear instructions and citation validation

Observability

- CloudWatch: latency, token usage, error rates
- Model invocation logging and Agent trace enabled
- CloudTrail for full audit trail

Cost Controls

- Cross-Region Inference (Global CRIS, ~10% savings)
- Right-sized models: Haiku for routing, Sonnet for generation
- Token budget limits in guardrails; query caching

Hands-On: Deploy and Test RAG App

01

Create Knowledge Base

Use the Bedrock console or SDK: S3 data source, Titan Text Embeddings V2, OpenSearch Serverless (quick create), fixed-size chunking 300 tokens / 20% overlap.

02

Deploy Serverless RAG API (Lambda)

Lambda retrieves from the KB with `HYBRID` search (top-5), assembles a grounded prompt with source tags, calls Converse API with guardrails, and returns answer + source URIs + token usage.

03

Test with Sample Queries

Five scenarios: (1) factual Q with citations, (2) out-of-scope Q → "I don't know", (3) prompt injection → guardrail block, (4) PII in query → masking, (5) multi-part question → completeness check.

04

Compare Approaches

Benchmark all four patterns side-by-side on your test queries and evaluate trade-offs in latency, control, and code complexity.

Approach	Pros	Cons
RetrieveAndGenerate (1 call)	Simplest, managed citations	Less prompt control
Retrieve + Converse (2 calls)	Full prompt control, custom logic	More code to maintain
Bedrock Agent	Autonomous, multi-tool, memory	Higher latency, more complex
Bedrock Flow	Visual, reusable, branching logic	Learning curve

Key Takeaways

Three RAG Patterns

Basic (managed, 1 API call), Custom (decoupled, full control), Agentic (autonomous ReAct reasoning)

Serverless is the Standard

API Gateway + Lambda + Bedrock gives you a scalable, low-ops foundation for production RAG

Agents & Flows Extend RAG

Agents add multi-tool reasoning; Flows add visual, branching, iterative orchestration with DoWhile loops

Multi-Agent Collaboration (GA)

Supervisor + specialist patterns enable complex, domain-partitioned workflows at scale

Built-in Evaluation (GA)

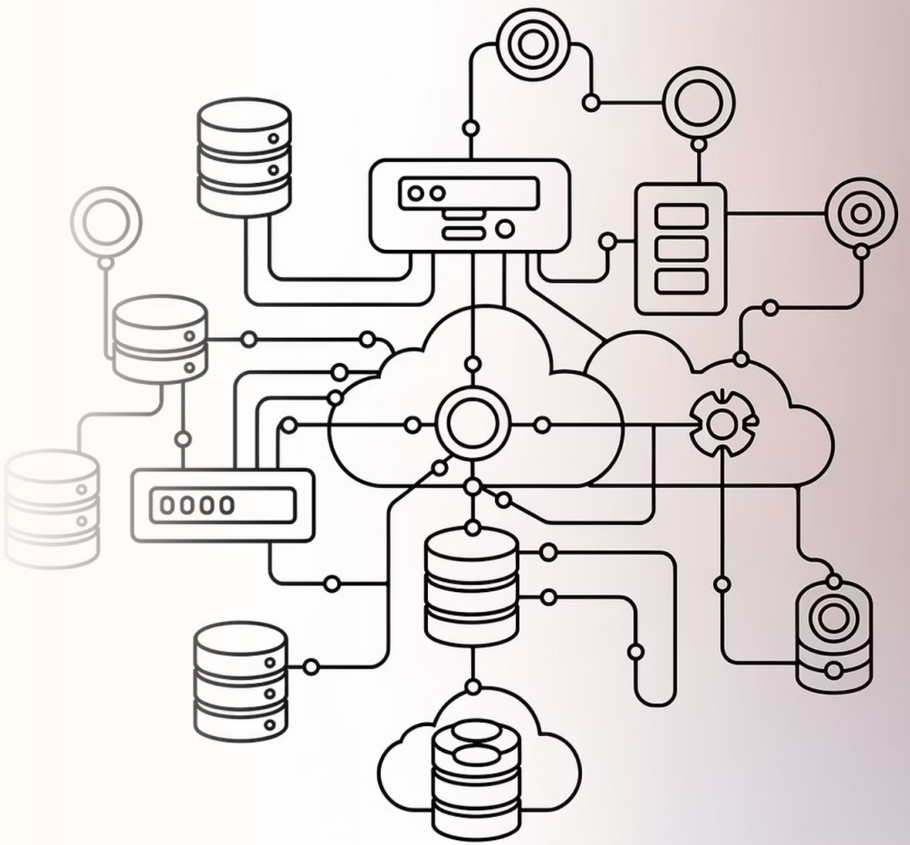
Measure retrieval, generation, and responsible AI quality with a single Bedrock API call

Production = Guardrails + Eval + Observability + Cost

Advanced techniques (HyDE, hierarchical chunking, Graph RAG) close the remaining quality gaps

Q&A

Cost, Security & Advanced Templates



Bedrock Pricing Model

On-demand pricing means you pay only for what you process — no upfront commitments required. Costs accrue per token, per query, and per evaluation job.

Core Billing Components

Component	How You're Charged
Model inference	Per 1K input + 1K output tokens
Embeddings	Per 1K input tokens
KB storage	Vector store costs (OpenSearch, S3, Aurora)
KB retrieval	Per Knowledge Base query
Guardrails	Per 1K text units evaluated
Model evaluation	Per evaluation job

Service Tiers

Tier	Latency	Cost	Best For
Priority	Fastest (+25% OTPS)	Premium	Customer-facing, latency-sensitive
Standard	Consistent	Regular	General workloads
Flex	Variable	Discounted	Batch, eval, non-urgent agentic
Reserved	Guaranteed TPM	Fixed monthly	Predictable high-volume

Global CRIS (Cross-Region Inference System) adds ~10% savings on both input and output tokens across all standard tier calls.

Cost Comparison: Model Selection Matters

Approximate on-demand pricing per 1M tokens (us-east-1, Standard tier). Choosing the right model is your single biggest cost lever.

Model	Input / 1M	Output / 1M	Best For
Nova Micro	$0.035	$0.14	Classification, intent routing
Nova Lite	$0.06	$0.24	Simple Q&A, visual tasks
Nova Pro	$0.80	$3.20	Balanced RAG generation
Claude Haiku 4.5	$0.80	$4.00	High-volume, fast responses
Claude Sonnet 4.5	$3.00	$15.00	Complex generation tasks
Claude Opus 4.6	$15.00	$75.00	Deep reasoning, orchestration
Llama 4 Maverick	$0.20	$0.85	Cost-effective general use
DeepSeek R1	$1.35	$5.40	Reasoning, math tasks

Key insight: There is a ~2,000x price difference between Nova Micro and Claude Opus on output tokens. Always choose the cheapest model that meets your quality bar — and validate with evals before committing. See current pricing at aws.amazon.com/bedrock/pricing.

Cost Optimization Strategies

1

Model Tiering

Route queries through Nova Micro (classify) → Nova Lite (simple) / Nova Pro (medium) / Claude Sonnet (complex). Use cheap models for routing, capable models only when quality demands it.

2

Prompt Caching

Cache repeated system prompts and context windows. Avoid re-processing identical tokens on every request — reduces both cost and latency significantly.

3

Token Budget Optimization

Set `maxTokens` appropriately — don't default to 4096 when 512 suffices. High maxTokens reduces concurrency (output tokens consume 5x quota via burndown rate). Monitor `OutputTokenCount` via CloudWatch.

4

Batch Inference

50% cheaper than on-demand for non-time-sensitive workloads. Ideal for evaluations, bulk summarization, and historical data analysis.

5

Service Tier Selection

Use Flex tier for evals and background processing. Reserve capacity for predictable high-volume production workloads.

6

KB & Distillation

Tune chunking to reduce irrelevant retrieval, use metadata filters, and manage sync frequency. Distill large model behavior into smaller, cheaper models — maintaining quality at lower inference cost.

Monitoring & Observability

CloudWatch Metrics — Automatic, No Extra Cost

Metric	What It Tells You
`Invocations`	Total API calls
`InvocationLatency`	End-to-end response time
`TimeToFirstToken`	Streaming latency (NEW)
`InputTokenCount`	Tokens sent to model
`OutputTokenCount`	Tokens generated
`EstimatedTPMQuotaUsage`	Quota consumption after burndown (NEW)
`InvocationThrottles`	Rate limit hits
`InvocationClientErrors`	4xx errors
`InvocationServerErrors`	5xx errors

Logging & Audit Tools

Model Invocation Logging

Logs full request/response payloads to CloudWatch Logs or S3. Disabled by default — enable for debugging and compliance audits. Use Athena + QuickSight for cost/usage dashboards.

CloudTrail

All Bedrock API calls logged (`InvokeModel`, `Converse`, agent calls). GuardDuty monitors CloudTrail for security anomalies. Essential for compliance audit trails.

Recommended Alarms

Set alarms at 70–80% quota utilization. Alert on throttling spikes. Monitor cost per day/week via invocation logs. Track `TimeToFirstToken` for SLA compliance.

Scaling Strategies

Bedrock provides multiple layers of scaling control. Combine them to build resilient, high-throughput architectures that protect against throttling and optimize cost.

Strategy	How	When to Use
Cross-Region Inference	Geographic or Global CRIS routing	Burst capacity, ~10% cost savings
Service Tiers	Priority / Standard / Flex per API call	Mixed workload optimization
Provisioned Throughput	Reserved model units (TPM)	Predictable high-volume production
Application Inference Profiles	Per-tenant / per-app tracking tags	Multi-tenant cost allocation
Lambda Concurrency	Reserved concurrency + SQS buffer	Protect against Bedrock throttling
API Gateway Throttling	Rate limits + usage plans	Protect backend from overload

High-throughput architecture pattern: API Gateway (throttle) → SQS Queue (buffer) → Lambda (concurrency=10) → Bedrock. SQS absorbs bursts; Lambda processes at a controlled, steady rate to stay within Bedrock quota limits — preventing cascading throttle failures.

Security Architecture

Data Protection

- TLS 1.2 encryption in transit for all API calls
- AWS KMS encryption at rest for KB, agents, models, and logs

Your data is **never** shared with model providers or used for training

- AWS PrivateLink (VPC endpoints) for private connectivity — no internet exposure

IAM Least Privilege

Restrict actions to specific models — never use `bedrock:*`

- Restrict to specific KB IDs and agent ARNs

Use `aws:RequestedRegion` conditions to limit regions

- Use SCPs to enforce guardrail usage org-wide

Guardrails as Security Layer

- Content filters: block harmful content (hate, violence, misconduct)
- Prompt attack detection: jailbreaks, injection, leakage
- PII masking: automatically redact sensitive data fields
- Contextual grounding: reject hallucinated responses
- Automated reasoning: validate against logical rules (99% accuracy)

Compliance Coverage

- ISO 27001, SOC 1/2/3, CSA STAR Level 2
- HIPAA eligible, GDPR compliant
- FedRAMP High authorized

Security Checklist for RAG Applications

🌐 Network

- VPC endpoints for Bedrock via PrivateLink — no public internet path
- Security groups restricting Lambda → Bedrock traffic only
- WAF on API Gateway for DDoS and injection protection

🔑 Auth & Authorization

- IAM roles with least privilege for Lambda execution
- API Gateway with Cognito or API key authentication
- SCPs preventing unauthorized model access across accounts

🔒 Data Protection

- KMS encryption on S3 buckets (documents + logs)
- KMS encryption on OpenSearch Serverless collections
- No PII in agent instructions or prompt templates
- Guardrails with PII filter enabled

📊 Monitoring & Audit

- Model invocation logging to encrypted S3
- CloudTrail enabled for all Bedrock API calls
- GuardDuty monitoring CloudTrail for anomalies
- CloudWatch alarms on throttles, errors, and latency

🛡️ Application Security

- Input validation before sending to Bedrock
- Guardrails with prompt attack detection enabled
- System prompts constraining model behavior scope
- Output validation before returning to end user
- Rate limiting enforced per user and per tenant

Cost Comparison Exercise

Scenario: A customer support RAG chatbot handling 10,000 queries/day (300,000 queries/month). Each query averages 2,000 input tokens (500 user query + 1,500 KB context) and 500 output tokens.

Model	Input Cost	Output Cost	Monthly Total
Nova Micro	300K × 2K × $0.035/1M = $21.00	300K × 500 × $0.14/1M = $21.00	**$42/mo**
Nova Pro	300K × 2K × $0.80/1M = $480.00	300K × 500 × $3.20/1M = $480.00	**$960/mo**
Claude Haiku 4.5	300K × 2K × $0.80/1M = $480.00	300K × 500 × $4.00/1M = $600.00	**$1,080/mo**
Claude Sonnet 4.5	300K × 2K × $3.00/1M = $1,800.00	300K × 500 × $15.00/1M = $2,250.00	**$4,050/mo**

96x Cost Gap

Nova Micro ($42) vs. Sonnet ($4,050). Is quality sufficient for your support use case?

Tiering Saves 50%+

Micro for routing + Pro for generation could cut Sonnet-level costs by more than half.

Add Infrastructure

~$50–200/mo for OpenSearch Serverless; ~$5–20/mo for S3 Vectors. Budget Guardrails per-text-unit.

Flex Tier Savings

Flex tier saves ~30% for non-real-time flows. Global CRIS adds ~10% savings on all token costs.

Security Best Practices Discussion

Use these prompts to drive a team conversation about threat modeling and security posture for your RAG application. There are no single right answers — context and compliance requirements will shape every decision.

What's the biggest security risk in a RAG application?

Consider: prompt injection, data leakage from source documents, hallucination presenting false information as fact, and unauthorized retrieval access.

How do you prevent verbatim source document exposure?

System prompt constraints, output validation, and Guardrails content filters can all play a role. What's the right combination for your use case?

PrivateLink vs. public endpoints — when do you need which?

Regulated industries and internal tooling typically require PrivateLink. Public endpoints are acceptable for low-sensitivity consumer apps with proper WAF and auth layers.

Multi-tenant data isolation in a shared Knowledge Base?

Metadata filters, separate KB collections per tenant, and IAM-scoped retrieval are the primary mechanisms. How do you balance isolation vs. operational overhead?

Guardrails vs. application-level validation — who does what?

Defense in depth: both layers are needed. Guardrails handle model-level threats; application validation handles business logic and upstream input sanitization.

How do you audit who asked what and what the model responded?

CloudTrail captures the API call; model invocation logging captures the full payload. Together they provide a complete, tamper-evident audit trail for compliance.

Key Takeaways

Model Selection is the #1 Cost Lever

Up to 96x price difference between cheapest and most expensive models. Always validate quality with evals before committing to a tier.

Tier Your Models Intelligently

Use cheap models (Nova Micro/Lite) for routing and classification; reserve capable models for generation tasks that require higher quality output.

Match Service Tier to Workload

Flex for batch and eval, Standard for general workloads, Priority for customer-facing latency-sensitive flows, Reserved for predictable high-volume production.

CloudWatch Metrics are Free — Use Them

Set alarms on throttles, latency, and token quota utilization at 70–80%. Enable invocation logging to S3 for cost dashboards via Athena + QuickSight.

Defense in Depth Security Posture

VPC endpoints + IAM least privilege + Guardrails + application-level input validation. No single layer is sufficient — all layers must work together.

Bedrock Meets Enterprise Compliance

HIPAA, GDPR, FedRAMP High, SOC 1/2/3, and ISO 27001 certified. Your data is never shared with model providers or used for model training.

Q&A

Connect

- https://github.com/eduamota/building-apps-with-bedrock
- https://www.linkedin.com/in/motaed/

- Agentic AI and Cybersecurity Risks (live online course with Petar Radanliev)
- AI Agents A-Z (live online course with Sinan Ozdemir)
- AI Memory Management in Agentic Systems (live online course with Richmond Alake)
- *Building Agentic AI Systems* (book)
- *AI Agents: The Definitive Guide* (book)


O'REILLY®